CSS Variables and CSS Functions

Css is an ever evolving language, new features are added to it all of the time. In recent years features have been added to make CSS have features like a traditional programming language like Javascript or C++. In this lab we will discuss CSS functions and CSS variables and how we can use them in a more sophisticated layout.

In this lab you will be working with the client Downward Dog Yoga.

Setup

Download *Lab-Yoga* and unzip it. It should contain a HTML document called *downward-dog-yoga.html*, a CSS document called *style.css* and also includes all of the related images. Load *downward-dog-yoga.html* into a code editor.

Take a moment to look over the HTML file and CSS file. *<header>*, *<main>* and *<footer>*.

Attached to the HTML document is a stylesheet that is laying out the the basic formatting including typography, colors and basic layouts such as width and heights. You notice that the layout is mostly using CSS grids.

CSS Functions

Functions are an important aspect of traditional programming. A function is a set of code that does a task. That task could be adding two numbers, writing a sentence or building a visual gradient. For most computer languages functions are written like this:

```
functionName( argument );
```

Writing the function names tells the device to run a set of tasks. For instance the *linear-gradient()* tells the browser to look for the protocol of building a linear gradient in the browser.

Sometimes functions need additional information to perform their task. This information is referred to as an *argument* is typically placed inside parenthesis. For instance for the background-image CSS property the *url()* function runs the tasks for looking for an external image file. The *argument* would be the name of the image file the browser should look for.

We can use functions in our code to do certain tasks, like adding or subtracting numbers, or choosing the larger or smaller of a set of numbers.

Calc()

The *calc()* function is short for calculation, meaning we can use it to math such add or subtracting. In the CSS document find the .fa-paw, this sets the size of the dog paw icon for the company. It is currently set to *24pt*, we can use the *calc()* function to subtract from it to make it smaller. Add the following:

```
.fa-paw {
  font-size: calc(24pt - 10pt);
}
```

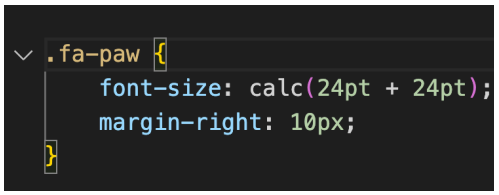




Save and preview in the browser. The result will be the value of the *font-size* will be *14pt* or smaller than it normal size.

We can do addition as well. Change the code to the following:

```
.fa-paw {
  font-size: calc(24pt + 24pt);
}
```





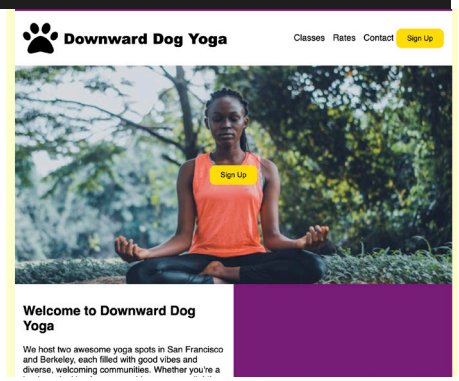
Save and preview. The result will be the value of the *font-size* will be *48pt* or larger than the normal size.

Min()

The *min()* function can be used to figure out the smaller of two values. In the CSS document find the *.container* class. Set the width property value to the following:

```
.container {
  ...
  width: min(800px, 1200px);
}
```





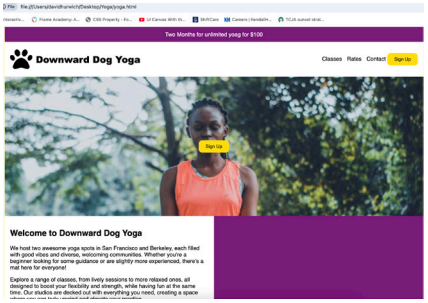
This is telling the browser to figure out which of the following numbers is smaller and use that as width value.

Save and preview in the browser. Since *800px* is less than *1200px* the browser uses that value and the webpage will appear thinner.

Max()

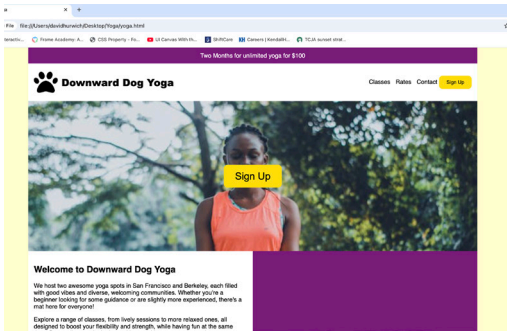


The *max()* function, like the *min()* function figures out the larger of two numbers.



Set the *.container* value from *min(800px, 1200px)* to *max(800px, 1200px)*. Save and preview in the browser, since 1200px is larger than 800px. the browser will use this value and the webpage will become wider.

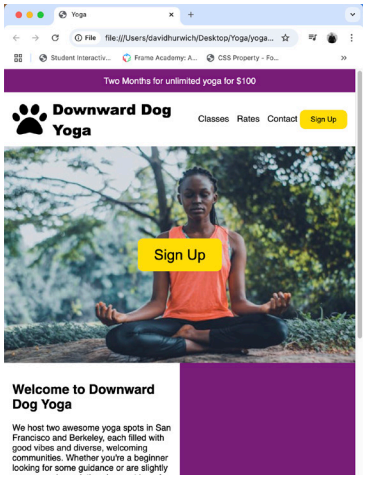
Using Calculations within Design



One thing about web development is the idea that designs need to work on large and small devices. At the moment our webpage is set to 1200px as it's width. If we preview the webpage in the browser if the browser is larger than 1200px the container is centered to the middle of the page. If we shrink the browser width eventually it will go below 1200px and part of the webpage will be cropped off.

We can use the min() function to assist us with this. Set the width of the .container to this:

```
.container {
   width: min(1200px, 100%);
     ......
}
```



What this means it will choose the lowest of the two to be the width of the .container. If the webpage is large, it will use the 1200px as the width and center it to the middle of the webpage. As the browser get smaller than 1200px the 100% becomes the lowest and the design becomes a bit more responsive.

CSS Variables

X = 5

1 + X

In math a variable represents a value that can change or can vary. In math if we know the value of a variable we can substitue that value into a mathematical expression. On the right since we know x is equal to *5* the expression beomes *1+5* or *6*. If the value of the x was not *5*, but instead *10* the expression would then become *1+10* or *11*.

What does this have to do with web development and CSS? When writing CSS we might reuse the same value many times such as a color or font. If we decide later we do not like a color we choose we would have to go into the code and change every instance of that color. With CSS variables we can establish a variable that represents a specific value. We can then use that variable in our CSS syntax. If that value of the variable changes, all instances of that variable are updated in the code.

Declare a CSS Variable

When creating a CSS variable it is referred to as "declaring." Declaring a variable is not a selector applying a property, so the syntax is different. Typically variables are declared to the *:root*, which is the base or root of the webpage or document object model.

```
:root {
}
```

To declare a variable you can come up with any variable name you want, just make sure:
starts with *"--"*

- no spaces
- no special characters
- Can not start with a number

If you view the webpage you'll notice that the purple in the Yoga website is a key color that is used very often. It is used as a *background-color*, *border-color* and text color. Let's give the variable name something descriptive like "*mainColor,*" since in terms of our design that's what it is. After declaring the variable of *--mainColor* we will use the existing value for the purple or *#7b1c7b*. Write the following:

```
:root {
 --mainColor: #7b1c7b;
}
```



Use a Variable

If we were to save and preview your webpage nothing will have changed, the reason being is that you declared a variable in your stylesheet, but you have not actually used it yet. This is a bit like creating an animation with *@keyframes*, but not calling it a selector.

Find the #special in the CSS document, this sets the visual style for the ad at the top of the website. You'll see that the *##7b1c7b* color is used as *background-color*. To use the *--mainColor* variable we declared use the function *var(variable name)*. *var()* will look for the value of a specific declared variable name. Since *var()* is a function it can use addition arguments or information, in this case the arguement or additionally information is the name of the variable, in this case *--mainColor*. The code would look like this:

```
#special {
  ...
  color: var(--mainColor);
  ...
}
```



Save and preview. You'll notice that the button color remains the same. What occured is the browser found the *--mainColor* variable and substituted it's value of *#7b1c7b* as the value for the font color.

Now that use *--mainColor* for the font color value. Find the #title section of the CSS document, note that the same color is being used as a font color. Let's use the same *var(--mainColor)* to set the font color to the purple, the code would look like this:

```
#title {
  ...
  color: var(--mainColor);
}
```



We can also apply the same *--mainColor* variable for the purple border color. Find header in the CSS document, note that the same purple color is being used in the *border-bottom*. Let's replace the purple with the *--mainColor* variable. The code would look like this:

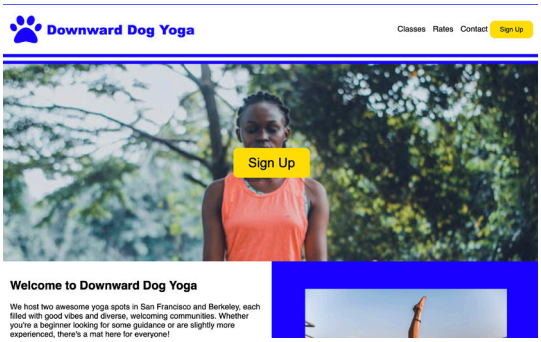
```
header {
  border-bottom: 20px double var(--mainColor);
}
```



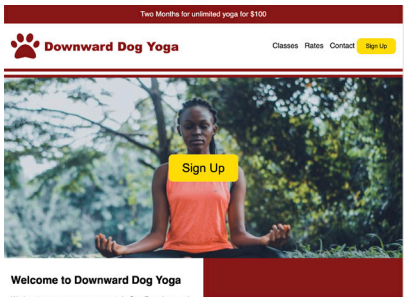
Use *var()* to replace every instance of *#7b1c7b* in the CSS code, this includes *#yoga-class* and *footer*.

Update the Variable

The client has looked over the webpage and is beginning to have second thoughts about using the purple color as the main color. Instead they are thinking of using a blue. Previously this would have meant finding every instance of the purple in the CSS code and replacing the value with the new value of blue. However, since our main color is set as a variable, all we need to do is update the value of the variable and the rest of the webpage will follow.



Return to *:root()* and change *--mainColor* from *#7b1c7b* to *"blue"* Save and preview you will see that all of the instance of the magenta has now been replaced with a blue. See how using variables made it so easy to update the color from yellow to blue.



The client has decided to go with a dusty orange-red of *#891818*. Change the *--mainColor* value to this value. When you preview you will see the design update to dark green.

Font Variable

CSS variables can be fonts as well. Return to *:root* and declare a variable called *--mainFont*. Set the value of *--mainFont* to *"Courier."*

```
:root {
  --mainColor: #073b00;
  --mainFont: “Times New Roman;
```



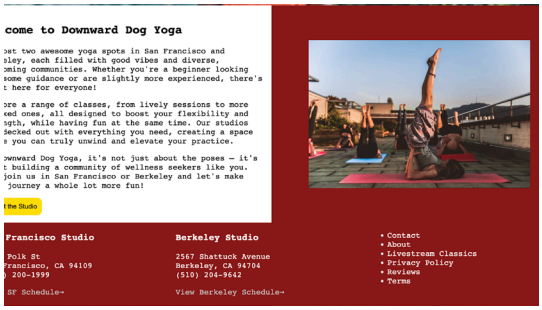
```
}
```

Find the header in the CSS document. Use var(--mainFont) to replace the value of font-family. It would look like this.




```
header {
    ...
    font-family: var(--mainFont);
}
```

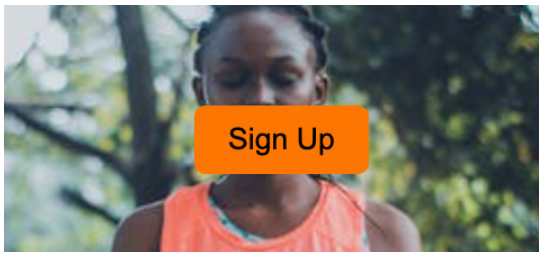
Save and you will notice that the font remains Times New Roman. Repeat the process for the main and footer CSS rules in the CSS document.



Once again, after client saw the webpage they decided instead of “Times New Roman” that they wanted to use “Courier”. Change the *--mainFont* value from *“Times New Roman”* to *“Courier”*. Save and you’ll see that all of the type updates as the --mainFont variable has changed.

Button Color Variable

Now that you understand the basics of CSS variables, try to make one on your own. In the *root()* create a css variable called buttonColor, set the value to *#ffde09*. Replace the value fo *#ffde09* within the CSS document to the new variable.

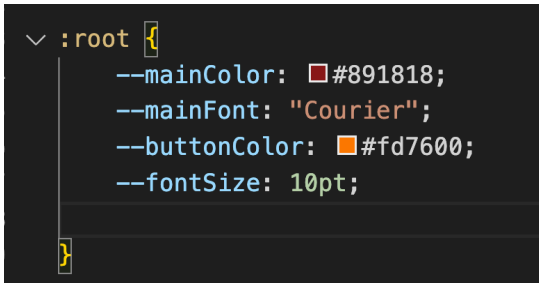


The client wants to change the *buttonColor* from yellow to light orange or *#ffcc55*

Variables and Functions

Let’s combine variables and functions together so that by changing the value of one variable we can affect a different element.

Let’s create a variable that represents the standard font. Go to :root{} and create a CSS variable called --fontSize and set the value to 10pt.




```
:root {
  ...
  --fontSize: 10pt;
}
```

Find .container in your CSS document and add in the property font-size and set the value to --fontSize variable.




```
.container {
   ...
   font-size: var(--fontSize);
}
```

Save and the general text should be setting the font to a size of 10pt.

Let's now set the company name to be three times the size of our *--fontSize* variable. How do we do that? By using the calc() function. Let's set the value of *#company-title* to a font size that a calculation of *fontSize* multiplied by three, here's how that would look.

```
#company-title {
   font-size: calc(var(--fontSize)*3);
}
```



What is going on above? First *var(--fontSize)* is grabbing the value of that variable, which is 10pt, and substituting that in as such

```
    calc(10pt * 3);
```



* means multiplication, the result is the calculation of 10pt x 3 or 30pt, thus the title of the company font size is now *30pt*.

Let's do the same thing with the dog paw icon. This size is being set by .fa-paw. Let's replace our original calculation of *v* with a new one based on our --fontSize variable. Let's set the font-size of .fa-paw to be 4.5 times the fontSize variable, it would look like this:

```
.fa-paw {
   font-size: calc(var(--fontSize)*4.5);
}
```





The result will be the icon is *45pt*.

Once again, after showing our design to our clients they thought the font size for the general text was too small, they want it larger, but also want the other text to scale proportionally as well. Change the value of fontSize from *10pt* to *15pt* and you will see that all of the text in the webpage scales with the new value.



Submission

Whew! That was alot and got a bit technical and tricky, but hopefully you have a better understanding of what CSS variables are and how to use CSS elements like functions to make your CSS more dynamnic..

Please turn all the related lab files to Canvas as a zipped folder for grading.